Lenguajes de Programación

PRUEBA FINAL SEGUNDA CONVOCATORIA

Fecha: 27 de junio de 2014

Valor de la prueba: 60%

EJERCICIO 1 (1 punto)

Sin utilizar funciones auxiliares, definir una función en lisp llamada **dividelista** que, dada una lista, devuelva una lista compuesta de dos listas, de forma que la primera contenga los valores en posiciones pares y la segunda los valores en posiciones impares.

```
> (dividelista '(1 2 3 4 5 6 7))
((1 3 5 7) (2 4 6))
> (dividelista '(2 1 1 2 3 3))
((2 1 3) (1 2 3))
> (dividelista '())
(NIL NIL)
> (dividelista '(1))
((1) NIL)
```

EJERCICIO 2 (5 puntos)

a) Implementa una clase llamada Utilidades que contenga un método que, dados como parámetros dos arrays y un número entero *k*, devuelva un nuevo array que contenga la suma elemento a elemento de los elementos del primer array con los elementos del segundo array invertido (el primer elemento del array resultado deberá ser la suma del primer elemento del primer array con el último del segundo; el segundo elemento del array resultado, la suma del segundo elemento del primer array con el penúltimo del segundo; y así sucesivamente). Para ello, utiliza un total de *k* hilos de forma que la carga esté equilibrada, teniendo en cuenta que podría ocurrir que la longitud de los arrays no fuera divisible por *k*. El método deberá devolver una excepción en el caso de que los arrays no tengan el mismo tamaño. Además, el tipo de los elementos del array deberá ser tan genérico como sea posible. Para ello, deberás definir las interfaces que sean necesarias.

b) Explica cómo podría utilizarse el método anterior para sumar en la forma descrita dos arrays de valores enteros.

.

EJERCICIO 3 (4 puntos)

Utilizando las herramientas jflex y java cup, proporciona las especificaciones necesarias para producir una clase que tome como entrada un listado de nombres y expresiones, y genere una clase que contenga un método que sea capaz de calcular cada expresión.

La primera línea del fichero de entrada contendrá la palabra class seguida del nombre de la clase que quiere generarse. Cada una de las líneas siguientes deberá contener un nombre seguido de los parámetros que se utilizan (entre paréntesis y separados por coma), el símbolo = y una expresión. La lista de parámetros no podrá ser vacía, y todos los parámetros y resultados se asumirán de tipo double. Los nombres deberán ser identificadores válidos en java. Las expresiones podrán contener paréntesis y los símbolos +,-,*,/. El orden de precedencia será el mismo que el del lenguaje java: * y / (con igual precedencia) y luego + y – (igualmente con igual precedencia). En el caso de operadores de igual precedencia, siempre se dará prioridad a la operación de más a la izquierda.

Se deberá generar una clase con el nombre especificado, y que contenga un método estático por cada una de las líneas que contienen un nombre y una expresión. Por ejemplo:

Entrada	Salida

class Prueba
operacion1(x,y)= (3*x)+y
cuadrado(x)=x*x
	public class Prueba {
	  double operacion1(double x, double y) {
	  return (3*x)+y;
	  }
	  double raizCuadrada(double x) {
	  return x*x;
	  }
	}

La clase generada deberá ser robusta a errores, e informar cuando la sintaxis del fichero de entrada no sea correcta. No es necesario comprobar que los identificadores utilizados en las expresiones se correspondan con los parámetros especificados.

En el caso de detectar un error, debe intentarse que el compilador se recupere lo antes posible.

```
;Sin definir funciones auxiliares, define una función llamada
dividelista que, dada una lista, devuelva una lista compuesta de
dos lista, de forma que la primera contenga los valores en
posiciones pares y la segunda los valores en posiciones impares.
; Ejemplos > (dividelista '(1 2 3 4 5 6 7))
; ((1 3 5 7) (2 4 6))
; > (dividelista '(2 1 1 2 3 3))
; ((2 1 3) (1 2 3))
; > (dividelista '())
; (NIL NIL)
;> (dividelista '(1))
; ((1) NIL)
(defun dividelista(l)
  (cond
     ((NULL l)  (LIST NIL NIL))
      ((NULL (cdr l)) (LIST (LIST (car l)) NIL))
      (T (LIST (cons (car l) (car (dividelista (cddr l)))) (cons
(cadr l) (cadr (dividelista (cddr l))))))
  )
 )
```

Hilos:

Mejor crear una clase sincronizada que devuelva el próximo archivo a procesar o null si no hay más. Cada hilo recibirá el objeto como parámetro y tendrá un bucle while con la condición de que el próximo archivo a procesar sea distinto de null. Hay que hacer el join en el método.